

CprE 487/587

Lab 2: CNN Model Implementation

“C makes it easy to shoot yourself in the foot; C++ makes it harder, but when you do it blows your whole leg off.” - Bjarne Stroustrup

1 Learning Objectives

By the end of this lab you should be able to

- Describe mathematically each layer of a basic Convolutional Neural Network (CNN).
- Implement basic model layers in standard programmatic syntax.
- Understand the required computations and nuances of low-level machine learning implementations.

2 Pre-lab

2.1 Lab Checklist

- Complete Lab 1 (or at least have a basic understanding of our model structure and each component layer).
- Have a basic understanding of C/C++.
- Have exported weights and intermediate feature map binaries from Lab 1 handy to test and debug your implementation.
- Access and download the lab data and templates. This can be accessed by the links provided on the Canvas assignment

2.2 Intro

In this lab, we are going to be implementing our own model layers in C/C++. We will be using the same model architecture as the `CNN_tinyimagenet.h5` from Lab 1, so this should be pretty familiar by now. These implementations will include a ReLU activation function, a deep/fully connected layer, a convolution layer, max pooling layer, and a flatten layer. Most of the custodial work is done for you, so you should be able to focus on the core layer operations in your implementation. Feel free to refactor the template code provided, but there should be no need to do so unless you are implementing additional functionality.

This lab is scheduled to take your group **one week** to complete.

3 Model Implementation

3.1 Given: A C/C++ DNN Template Framework

For this lab, we will be utilizing a template framework for you to implement the primary function of our image classification model. This should expedite the process of developing and testing your implementations and hopefully remove ~~much~~ some of the frustration. You will find layer C++ template files in the `src/layers` directory. The overall model will be constructed in `src/ML.cpp`. You may search for `TODO` to find where you should edit the files. **For more information on how to compile and run the framework on both your local x86 machine and the ZedBoard please see the README in the repo.** While the framework should require no changes outside the operations you will implement, feel free to tinker with it and make changes. If you do so, please describe those in your lab document (e.g, Why did you make the change? What differences did you see in performance or implementation following the change?) and include the source code that was altered.

3.2 Layer Operations

As a first design step, you should identify all the required layers and their mathematical function. So, for the neural network model used in Lab 1, **list the different types of layers present and write the mathematical formulation of each layer type.** When writing the mathematical notation, use the *tensor index notation* introduced in the textbook (see equations 2.2 and 2.3). Provide the specific values of the relevant (to a particular layer) parameters listed in Table 2.1 of the textbook. **Also include a brief description of each variable, its use/purpose, and how that is represented in our model.**

Once you have a firm understanding of the requisite layers, **write a C++ implementation for each layer needed our model.** Use the provided template! Do not try to over optimize your solution – simply write a straight-forward implementation, focusing on correctness and readability.

For testing purposes, use the binary data exported from Lab 1 for input feature maps, weights, and output feature maps. Compare functions are provided for you to compare your operation output to that of the Tensorflow implementation (see `compare*` methods of `LayerData` in `src/layers/Layer.h`). These functions compare the output of a layer elementwise with the expected output from Tensorflow. It is vital that you use these compare functions (or something similar) for your operation verification. **Report the result of appropriate comparison functions and justify that the value is acceptable.**¹

3.2.1 Operations/Layers

- **Fully Connected Layer** A fully connected layer is one in which all nodes are connected to all inputs from the previous layer. Each “*connection*” has an associated weight which is multiplied by the input value prior to summing at the node. A bias is then added before the value is used as input to an activation function, ReLU in our case.
- **Convolutional Layer** Convolution is the process of combining multiple local inputs by multiplying them element-wise by a filter/kernel and then summing the multiplied results to produce an output value. This process can be thought of as a sliding window across the input by the weights and then directly mapping to a single output value with each

¹I.e., you should not directly compare values by hand to those outputted by Tensorflow. Each layer has 1,000's of values and repeatedly comparing them by hand will be tedious and error-prone.

operation. The size of the filters/kernels can vary, the stride or distance the window moves with each operation, and the number of kernels per layer can vary. Ultimately, it is a function that quantifies the similarities between the input feature map subsections and the features represented by the filters/kernels. In image classification, this then can be thought of as a filter representing an important pattern determined in the training process (eyes, diagonal line, circle, square, etc.) that is used to classify an image.

$$ReLU(g) = \begin{cases} 0 & g < 0 \\ g & g \geq 0 \end{cases}$$

Figure 1: ReLU activation function mathematical formula.

To match TensorFlow’s layer terminology, your convolutional layer should include the activation layer. For your case, this is **ReLU Activation Function**. A rectified linear unit function is a function that matches the equation in Figure 1.²

- **Max Pool**
- **Softmax Activation Function** For a vector of N inputs $g = (g_0, \dots, g_i, \dots, g_{N-1})$ the softmax $Y_i(g)$ is defined as

$$Y_i(g) = e^{g_i} / \sum_{l=0}^{N-1} e^{g_l}$$

3.2.2 Tips

- Do not procrastinate on this lab. While you are granted two weeks for this lab, it can be more challenging than first impressions suggest. Random bugs can set your group back and there are multiple facets of this lab to complete. Take this into consideration when planning your group’s time allocations and meeting times outside lab. Ultimately, this lab represents the base for future labs!
- Use [Visual Studio Code](#) installed on your machines. VSCode offers many handy features such as live “Google Docs-like” sharing with [Live Share](#). This allows all your teammates to work in tandem and see the code at once. C++ Debugging is also a breeze with their GUI debugger built in. See this [article](#) or your TAs for additional details on setting this up and usage. You may also use your favorite editor, although you may have additional difficulty in sharing with your teammates and debugging.³
- Utilize Git (either our [Gitlab](#) instance or [Github](#)) to save and share your code. Sharing over Google Docs, One Drive, U Drive, or Flash Drive (or even worse, one partner having the only copy on their desktop! 🙄) is discouraged. Too many times these methods have caused issues for students in the past. A small amount of time spent in managing your code can save you a lot of headache later.
- Develop each operation individually. Do not let errors in one layer/operation affect your results of the others. While we do expect that you pull all your layers together in the end and have each successive layer feed into one-another, feed your exported intermediate feature

²See this [article](#) for an in-depth review of the ReLU function and its significance in machine learning.

³Other IDEs and build processes not 100% supported.

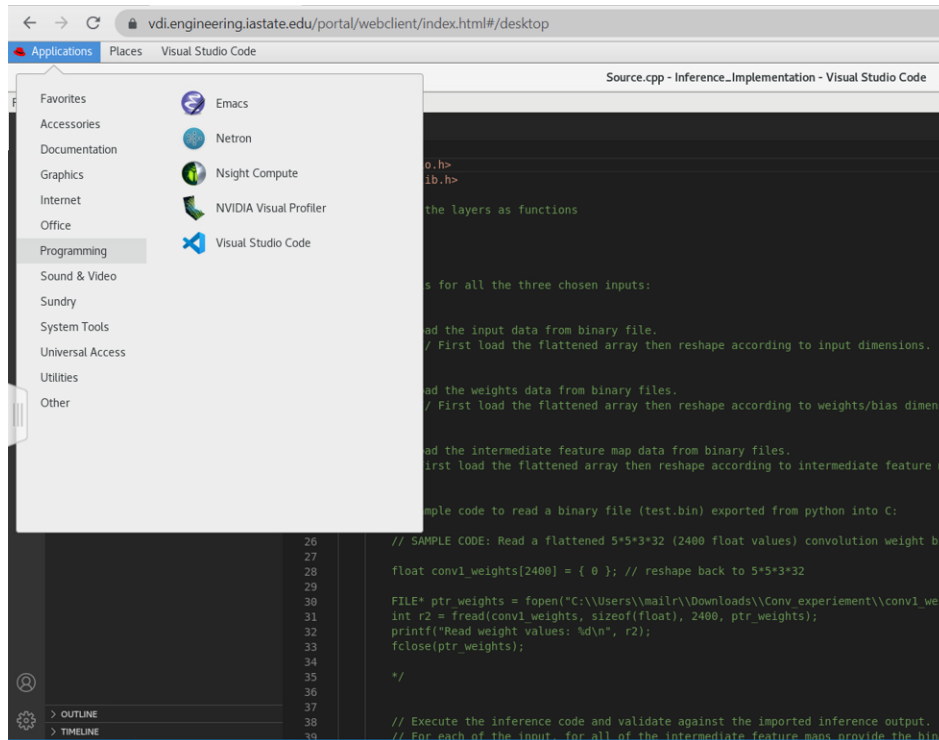


Figure 2: Visual Studio Code location of RHEL Coover 2050 lab computers.

maps into your layers and compare that with the result generated by Tensorflow. Once you are confident in your implementation, then work on combining them.

- It is important to understand exactly what your code is doing. A average *unoptimized* solution should take at most 120 seconds to run the entire model (all layers) on the lab machines.
- While it is unlikely you will encounter large array allocation when utilizing the provided framework, it is recommended that you use dynamic allocation methods (*malloc + free & new + delete*) for this lab. Dynamic allocations are placed on the heap at run time compared to static allocations (*int a[100]*) placement on the stack at compile time. With large arrays, it is possible to overflow our program's virtual stack without special compile arguments that can cause magic errors that can be hard to detect. If you are getting odd crashes, double check this.⁴
- Some error is likely in your solution – plan for time spent debugging!

3.3 Analysis

1. Estimate the size of the memory needed for each layer and how many MAC computations are required for each layer.
2. Verify that your outputs match the expected outputs sourced from the Tensorflow implementation. If you are unable to fully complete all components, include a write-up about the progress you made, any issues you encountered, and what next steps you would have taken to tackle the problem given more time.

⁴See this [article](#) & [Stack Overflow post](#) (no pun intended) for further information.

3. What was the final maximum error you saw/the error tolerance you used in your verification process for comparing your output? How did you come to that value and what is a reasonable error range? Where does this error come from and should there be any at all?

3.4 Profiling C++ Implementation

Now profile your baseline C/C++ TinyImageNet inference implemented as a part of Lab 1. Note that this part assumes that you have implemented all the layers end-to-end and are executing them for a given input. Perform the following steps:

1. Execute your inference application and report the time taken for a single inference on both your x86 lab machine and the ZedBoard. Compare these to the time taken when a single inference is executed using the Tensorflow backend. You could use Tensorboard to measure the performance. If you see a difference in execution time, why do you think there is a difference?
2. Now using perf (for x86 as described in Section 3.5) and the Timer class (for the ZedBoard in src/Utils.h), figure out which layers takes most of the computation time. Plot the layerwise execution time fraction and report it. State possible reasons why a particular layer takes significant computation time? Does this match your Lab 1 results from Tensorboard? Which layers may be ripe for optimization? Why?

3.5 Using Perf for Benchmarking

3.5.1 Compiling C++ & benchmarking using time

Now having a working baseline implementation of TinyImageNet inference in C/C++, we would first try to get familiar to compile and use it on a linux machine. This part could be performed in either the local machine in Coover 2050 or on the VMs used for training. Whichever you use, note the hardware configuration and keep it consistent for comparisons. For editing you may use the VScode application. For compiling and executing your code, you may use the following steps (e.g. if your main file is Source.cpp):

1. Navigate to the directory of the file. For compilation use the use g++ compiler located at /usr/bin/g++. You can compile your code by executing the following command in the terminal: `g++ -o app Source.cpp`
2. If for the later part of the lab you use the pthread library for multithreading and math library functions such as exponential for implementing softmax. You must compile your code by executing the following command: `g++ -o app Source.cpp -lpthread -lm`. For compiler to be successfully able to link those libraries.
3. Execute the code by running `./app` executable
4. You can approximately benchmark the execution time of your application by using time and running the following command: `time ./app`
5. In the output, make sure to use the real time ("overall time spent in the program as if there was a real human with a stopwatch, this may still include time spend by other processes but it is reasonable")

3.5.2 Profiling using perf

Perf is a tool that uses CPU performance counter registers to perform lightweight application profiling. More information on it could be found at the link: [Perf_Wiki](#).

For example, to profile an executable `app` using `perf`, you would execute the following command to generate the profile data: `perf record --count=100 ./app`. Please take care to keep a reasonable value for count because the less the count the more frequently perf does the sampling but the profiling overhead increases. Then you could see the breakdown of the runtime cost of your application using the command: `perf report`.

4 Above & Beyond 🚀

- Write a script to determine top-1, 5, and 10 accuracy when running on ZedBoard. Even though your layer errors may be very small, this may be different from your tensorflow / python implementation.
- Add a new layer to our network (e.g., Normalization, Dropout, etc.) with a new operation and implement that in C++ (This may require re-training our model in python in order to get the learned weights from Tensorflow).
- Implement a new activation function for our model and describe situations in which your activation function may improve performance (from either a hardware or inference perspective). Bonus if you re-train our model specifically with the new activation function and compare it to the ReLU implementation.
- Re-implement our model in a language aside from C++. C++ gives us a base platform for low-level operations, but by no means is the only way to do so. Compare the performance and implementation complexity to that of our framework.

5 Submission Instructions 📁

Below is a list of the required turn-in components for the lab to be submitted to Canvas. Only the final submission will be graded, although you are free to resubmit as many times as you please. All submissions must be complete. Even slight changes require a complete re-submission, **NOT** a partial, single file, change. Naming conventions for each file are also provided below. <Lab Partners Number> represents the number in brackets from your Canvas group's name.

5.1 Turn-In Checklist

- PDF of your lab report, either from the template or your own format containing all the required lab components (`lab2_report_<Lab Partners Number>.pdf`).
- Final source code directory for your implementation (`lab2_src_<Lab Partners Number>.zip`).

5.2 File Structure

Each Canvas submission:

```
├─ lab2_report_<Lab Partners Number>.pdf
└─ lab2_src_<Lab Partners Number>.tgz/zip
```

6 Appendix: Beta Framework Setup

6.0.1 Configuring the Zedboard

We also need to ensure that the Zedboard's physical jumper pins are properly set to allow for use of the SD Card reader. Below is a list of the physical pins that should be altered from the default state. All other values should be kept default as per the [Zedboard Hardware User Guide](#) on page 35/36. A diagram is also provided below for reference. If your board is not properly configured, please notify your TA so that ETG can properly configure the boards for use and switch to a different board if needed.

- JP2: Open (connect 5v output to USB OTG - not used)
- JP3: Open (USB vBus capacitor - not used)
- JP6: Shorted (SD Card Operation)
- JP7: Shorted (SIG - GND)
- JP8: Shorted (SIG - GND)
- JP9: Shorted (SIG - GND)
- JP10: Shorted (SIG - GND)
- JP11: Shorted (SIG - GND)
- JP18: Shorted (2V5)
- JP21: Open (Current sense)

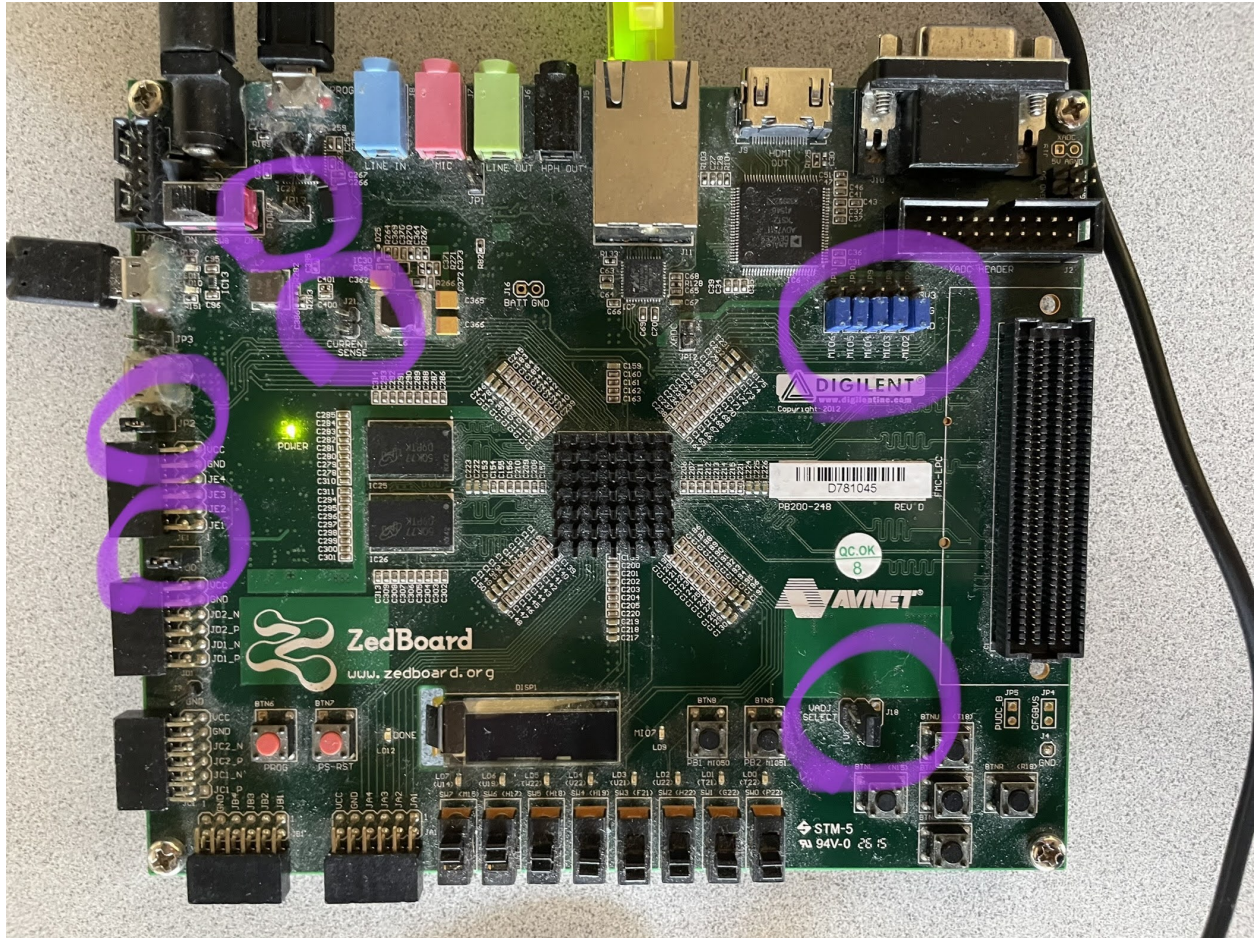


Figure 3: Physical Zedboard Jumper Configuration.

6.0.2 Flashing the SD Card

Finally, we need to flash the SD cards. If they have not already been done for you, they can be done on any computer with capabilities to read and write to SD cards. Ensure the SD card is at least 1 GB in size or larger. Re-partition the SD card with a single partition utilizing the entire SD card capacity if needed. Format the SD card using the FAT32 file-system (exFAT should also be supported, but is currently untested) using the default page-size (tested working with 16k). Once that completes, then you can copy the files in the data folder of the framework along with any other files onto the SD card. NOTE: there is a max filename/directory size/length of 255 characters (overflowing this will cause a non-descriptor *Cannot open file error*).